Universidad Nacional Autónoma de México

Facultad de Ingeniería

Modelos de Programación Orientada a Objetos

Profesor(a):

Prof. Oscar Manuel Ruiz Hurtado.

Semestre:

2027-1

Practica: 9

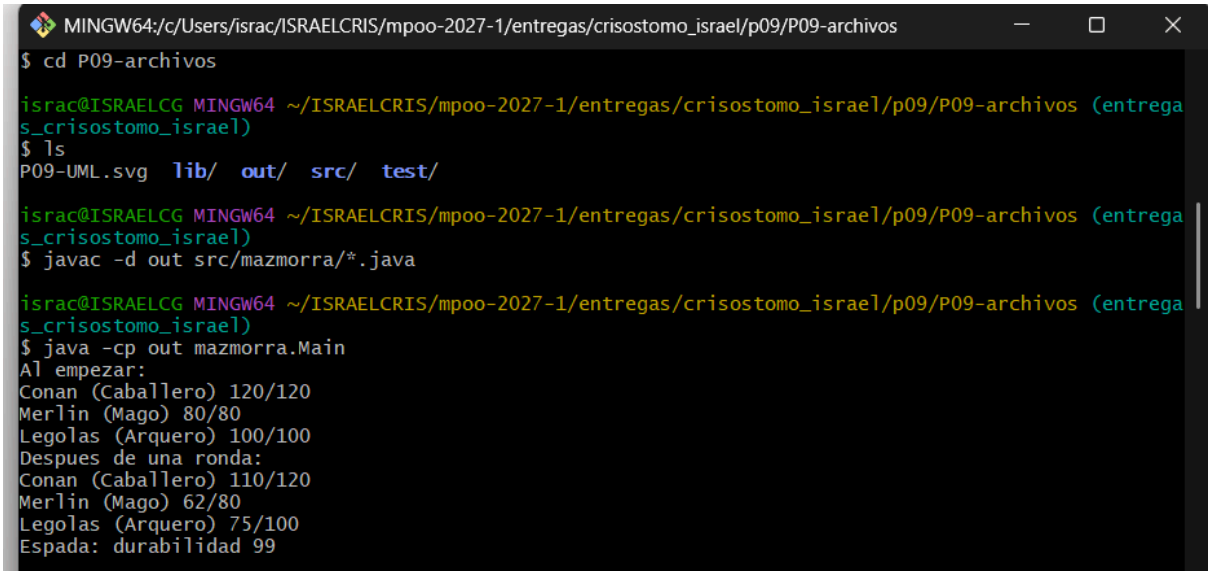
Grupo: 6

Integrantes:

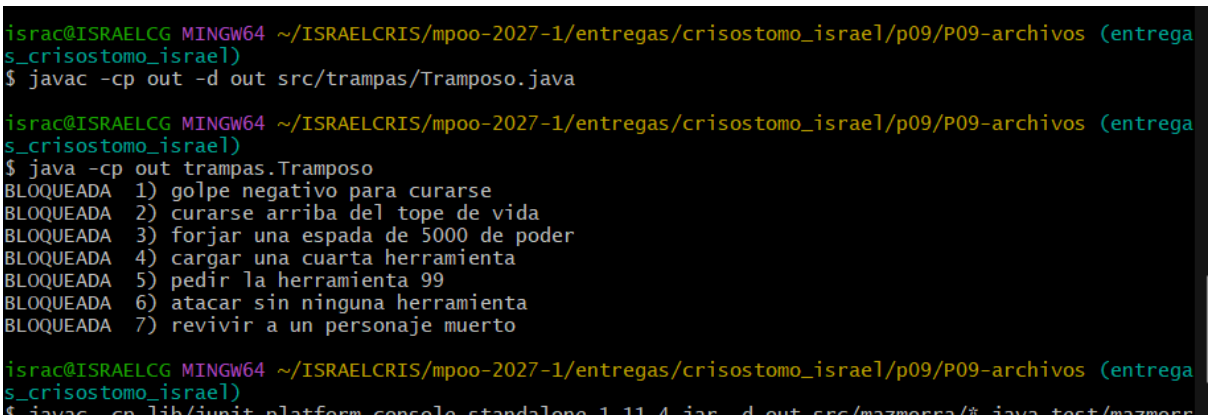
Crisostomo Garcia Israel

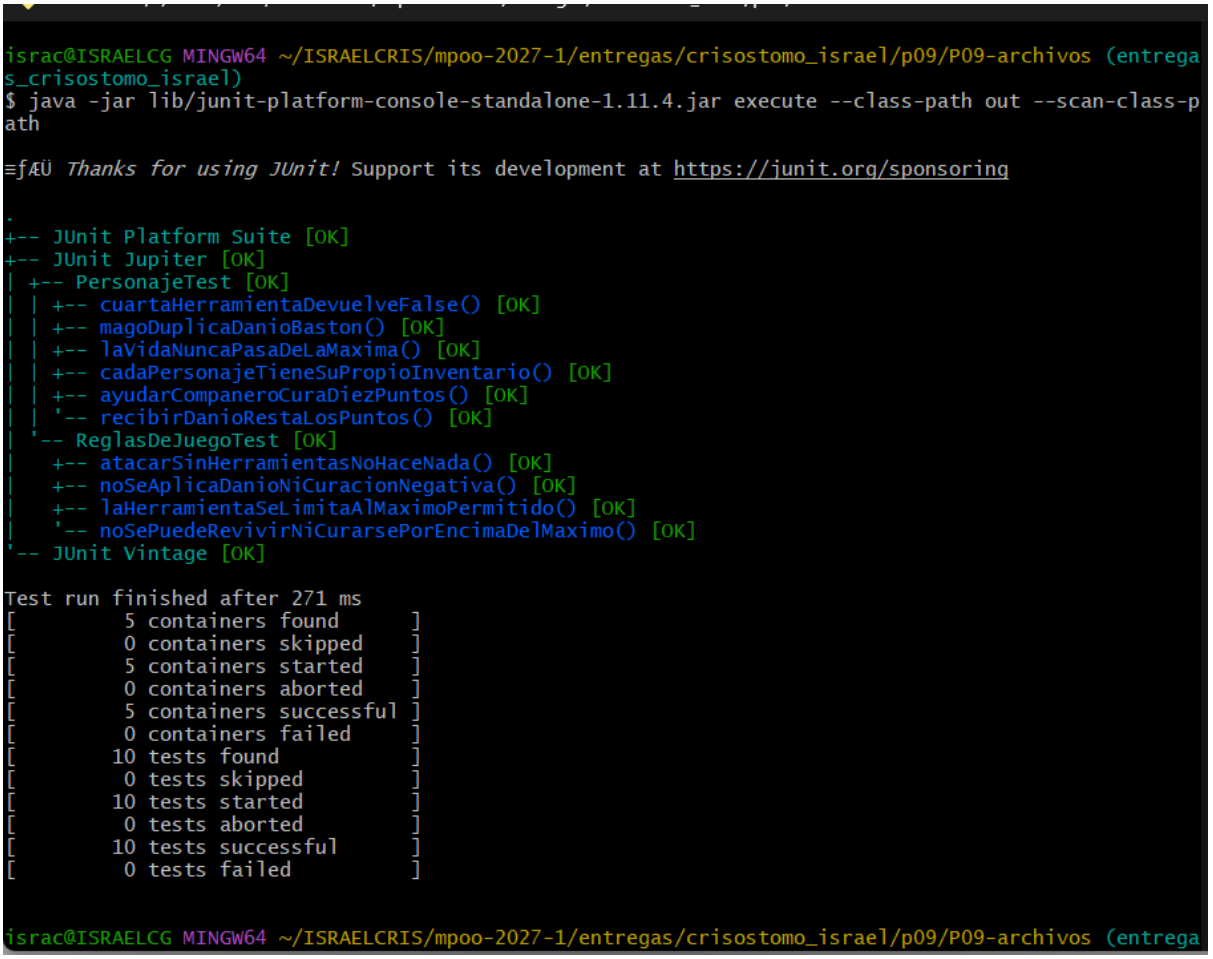
Mendez Reyes Juan Diego

Villarruel Sánchez David Uriel


MINGW64:/c/Users/israc/ISRAELCRIS/mpoo-2027-1/entregas/crisostomo_israel/p09/P09-archivos
$ cd P09-archivos
israc@ISRAELCG MINGW64 ~/ISRAELCRIS/mpoo-2027-1/entregas/crisostomo_israel/p09/P09-archivos (entregas_crisostomo_israel)
$ ls
P09-UML.svg lib/ out/ src/ test/
israc@ISRAELCG MINGW64 ~/ISRAELCRIS/mpoo-2027-1/entregas/crisostomo_israel/p09/P09-archivos (entregas_crisostomo_israel)
$ javac -d out src/mazmorra/*.java
israc@ISRAELCG MINGW64 ~/ISRAELCRIS/mpoo-2027-1/entregas/crisostomo_israel/p09/P09-archivos (entregas_crisostomo_israel)
$ java -cp out mazmorra.Main
Al empezar:
Conan (Caballero) 120/120
Merlin (Mago) 80/80
Legolas (Arquero) 100/100
Despues de una ronda:
Conan (Caballero) 110/120
Merlin (Mago) 62/80
Legolas (Arquero) 75/100
Espada: durabilidad 99


israc@ISRAELCG MINGW64 ~/ISRAELCRIS/mpoo-2027-1/entregas/crisostomo_israel/p09/P09-archivos (entregas_crisostomo_israel)
$ javac -cp out -d out src/trampas/TrampasDeAcceso.java
src\trampas\TrampasDeAcceso.java:10: error: vida has private access in Personaje
heroe.vida = 9999;
^
src\trampas\TrampasDeAcceso.java:11: error: inventario has private access in Personaje
heroe.inventario = new Inventario();
^
src\trampas\TrampasDeAcceso.java:14: error: durabilidad has private access in Herramienta
espada.durabilidad = 1000000;
^
src\trampas\TrampasDeAcceso.java:17: error: herramientas has private access in Inventario
bolsa.herramientas[0] = espada;
^
src\trampas\TrampasDeAcceso.java:18: error: cuantas has private access in Inventario
bolsa.cuantas = 50;
^
5 errors
israc@ISRAELCG MINGW64 ~/ISRAELCRIS/mpoo-2027-1/entregas/crisostomo_israel/p09/P09-archivos (entregas_crisostomo_israel)


israc@ISRAELCG MINGW64 ~/ISRAELCRIS/mpoo-2027-1/entregas/crisostomo_israel/p09/P09-archivos (entregas_crisostomo_israel)
$ javac -cp out -d out src/trampas/Tramposo.java
israc@ISRAELCG MINGW64 ~/ISRAELCRIS/mpoo-2027-1/entregas/crisostomo_israel/p09/P09-archivos (entregas_crisostomo_israel)
$ java -cp out trampas.Tramposo
BLOQUEADA 1) golpe negativo para curarse
BLOQUEADA 2) curarse arriba del tope de vida
BLOQUEADA 3) forjar una espada de 5000 de poder
BLOQUEADA 4) cargar una cuarta herramienta
BLOQUEADA 5) pedir la herramienta 99
BLOQUEADA 6) atacar sin ninguna herramienta
BLOQUEADA 7) revivir a un personaje muerto
israc@ISRAELCG MINGW64 ~/ISRAELCRIS/mpoo-2027-1/entregas/crisostomo_israel/p09/P09-archivos (entregas_crisostomo_israel)



1. Las cuatro relaciones: para herencia, composición, agregación y asociación, copia la línea de tu código donde se ve cada una y di qué le pasa a la parte si el dueño desaparece (para la herencia, qué hace **super(...)**).

herencia:

```
public class Caballero extends Personaje {
    public Caballero(String nombre) {
        super(nombre, 120);
```

la herencia utiliza “es un ” y al usar la instancia de Caballero , se crea un objeto y se guarda en la memoria y contiene las características de padre Personaje,o cuando llamamos a “`super(...)` ” es obligatorio porque es la que garantiza que el núcleo

de personaje se pueda construir antes de que se agreguen las especificaciones del caballero

Composición: (◆ **rombo lleno**)

```
this.inventario = new Inventario();
```

Representa una relación de pertenencia fuerte ("tiene un" de forma exclusiva). El `Personaje` es el responsable absoluto de fabricar su propio `Inventario`. Al usar la palabra reservada `new` dentro del constructor y guardarlo en un atributo `private final` Si el personaje muere y es destruido por el sistema, su inventario muere exactamente en ese mismo instante, ya que nadie más en el programa tiene acceso a él.

Agregación (◇ rombo vacío)

`conan.equipar(espada);` llamado desde `Main`, que a su vez se conecta con `public boolean agregar(Herramienta h)` en la clase `Inventario`.

Es una relación de pertenencia débil ("usa un"). La `Herramienta` (como la espada) no fue fabricada por el personaje ni por el inventario; fue creada de forma independiente en la clase `Main` y luego fue "prestada" o "pasada" al inventario a través del método `equipar`. Debido a esto, el ciclo de vida de la herramienta es totalmente independiente.

Asociación (línea simple)

`merlin.setCompanero(conan);` en `Main` y el atributo `private Personaje companero;` en `Personaje`.

Es una relación estructural entre dos objetos independientes que deciden "conocerse" o interactuar temporalmente. En términos de diseño, esto implica un "acoplamiento débil". Merlín tiene una referencia hacia Conan para poder curarlo, pero Merlín no es "dueño" de Conan.

2. Los huecos: ¿alguna trampa funcionó la primera vez? ¿Qué cambiaste? Si ninguna funcionó, elige una y explica qué línea de tu código la bloquea. ¿Por qué el diagrama no tiene **setVida** ni **getHerramientas.**

Bloqueo de la trampa 3 (Forjar una espada de 5000 de poder): Esta trampa fracasó gracias a la validación estricta en el momento de la instanciación. La línea que bloquea esto en el constructor de `Herramienta` es:`this.danio = entre(danio, 1, 50);`**Justificación:** Un buen encapsulamiento no solo oculta los datos, sino que protege la "integridad" del objeto. Aunque un jugador tramposo intente forzar un daño de 5000 enviándolo como argumento, el objeto `Herramienta` se defiende a sí mismo pasándolo por el método privado `entre()`. Este método utiliza `Math.min` y `Math.max` para acotar cualquier número fuera de los límites y forzarlo a un máximo de 50. Además, al declarar el atributo como `private final int danio;`, garantizamos que una vez creado, sea inmutable: ni siquiera el propio objeto podrá alterar este valor más adelante.

Por qué el diagrama no tiene `setVida` ni `getHerramientas`?

la falta de de `setVida`: En la Programación los objetos no deben ser simples "contenedores de variables" que cualquiera pueda modificar a su antojo mediante *setters*. Si existiera `setVida(int nuevaVida)`, un tramposo podría invocar `heroe.setVida(9999)`. Al omitirlo, obligamos a que el estado interno cambie únicamente mediante métodos de "lógica de negocio" (`recibirDanio` y `curar`).

y en el caso de la ausencia **`getHerramientas`:** Si tuviéramos un método que devolviera el arreglo (ej. `public Herramienta[] getHerramientas()`), estaríamos devolviendo la "referencia de memoria" del arreglo original. Esto crearía una fuga de encapsulamiento gravísima. Un jugador podría obtener el arreglo y sobreescribir un espacio directamente (`inventario.getHerramientas()[3] = nuevaArma;`), ignorando por completo el límite de 3 herramientas y saltándose las validaciones del método `agregar()`

3. Una prueba roja: copia el mensaje del paso 4b y explica qué significa cada número y quién estaba mal.?

¿Qué significa cada número en la filosofía de pruebas?

- `<70>` (Valor esperado): Es la hipótesis de quien escribió la prueba. El programador (o tester) redactó `assertEquals(70, p.getVida());` asumiendo por inercia que todos los personajes empezaban con un estándar de 100 puntos de vida (100 de vida base - 30 de daño = 70 restantes).
- `<50>` (Valor real): Es el resultado verídico y objetivo que arrojó la ejecución del código tras aplicar las matemáticas del sistema.

quién estaba mal.?

Según las reglas del diagrama UML, la clase `Mago` se inicializa con `super(nombre, 80)`, es decir, es más frágil y su vida máxima es 80. Al restarle los 30 puntos de daño, el resultado correcto es 50. Esta situación demuestra un principio fundamental de JUnit: *una prueba en rojo no siempre significa que el código fuente esté roto o mal programado*. A veces, significa que la prueba misma fue diseñada ignorando las reglas del negocio. Para solucionarlo y volver a obtener una prueba en verde, la aserción debe corregirse alineándose a la realidad del diseño: `assertEquals(50, p.getVida());`.